

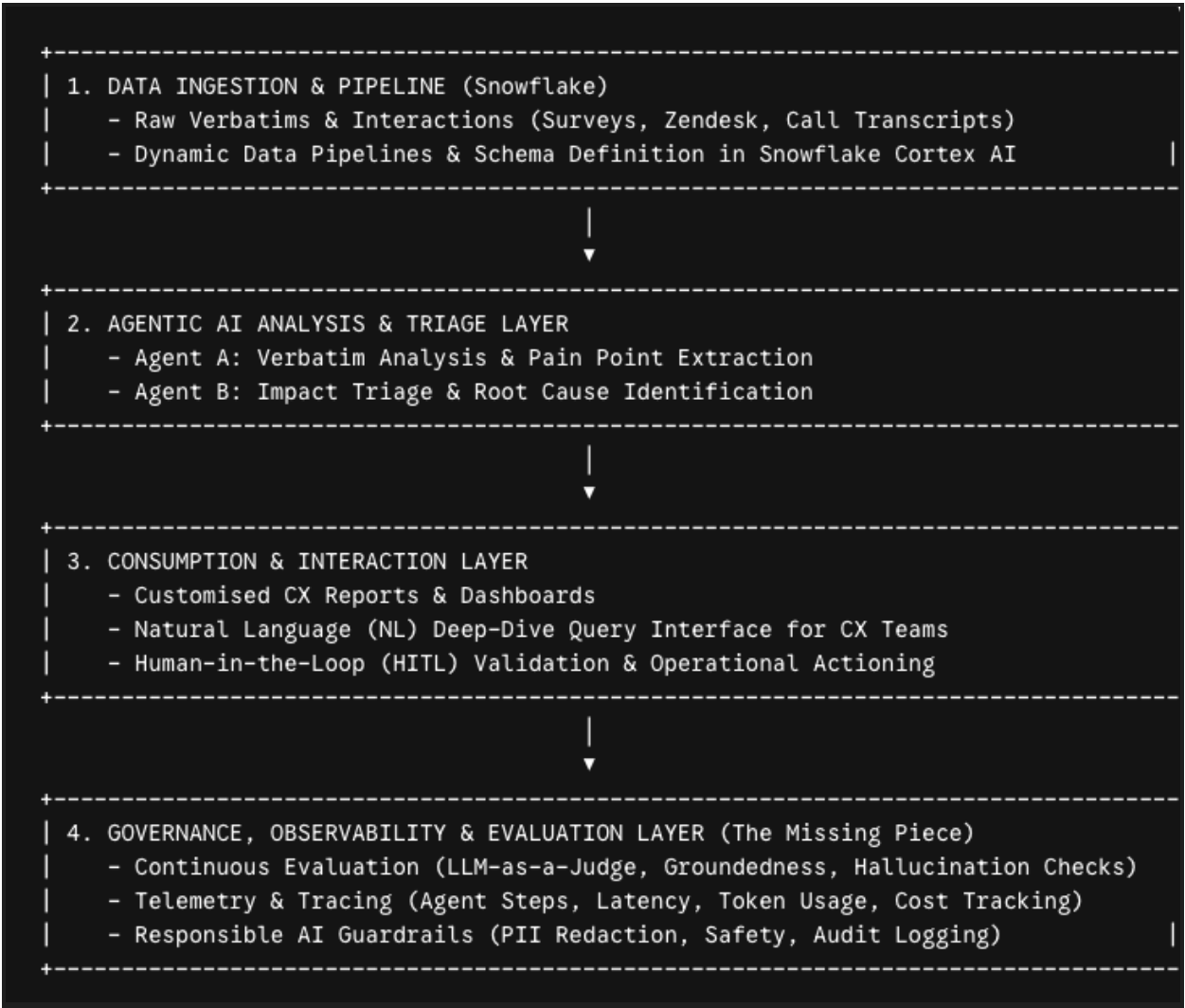
Agentic AI & Intelligent Workflows: Designed and delivered an end-to-end Agentic AI solution using multi-agent orchestration to synthesise complex member interactions into actionable insights and prioritised actions. Embedded human-in-the-loop controls to support safe adoption and accelerate knowledge-intensive CX workflows, with a projected 5-point increase in NPS.

I'll help you structure this into a clear architecture layout and an executive narrative to showcase how Snowflake, Cortex AI, multi-agent orchestration, and human-in-the-loop (HITL) controls work together end-to-end.



End-to-End System Architecture

Here is how the data, AI agents, and human governance flow through the pipeline:





The Narrative (Executive Summary)

Here is a narrative framework you can use for interview discussions, presentation slides, or case studies:

Objective & Scope:

To transform unstructured member verbatims and feedback into actionable operational insights, I designed an **Agentic AI architecture** integrating **Snowflake Cortex AI** with a **multi-agent orchestration framework** and **Human-in-the-Loop (HITL)** controls.

1. Data Engineering & Schema Definition (Snowflake & Cortex AI):

Unstructured interaction data (such as call logs, survey responses, and service tickets) is ingested into **Snowflake**. Using Snowflake data pipelines and structured schemas defined within **Cortex AI**, the raw text is parsed and prepared for downstream LLM analysis.

2. Agentic Orchestration & Root-Cause Triage:

Multi-agent workflows automate complex analytical tasks:

- **Verbatim Analysis Agent:** Scans feedback text to detect sentiment, extract key themes, and surface specific member pain points.
- **Impact Triage & Root Cause Agent:** Evaluates extracted issues against historical interaction data to score member impact, categorizes priority, and identifies underlying root causes.

3. Human-in-the-Loop Validation & Actionability:

Before automated recommendations are pushed to downstream operational systems, the **Customer Experience (CX) Team** reviews, validates, and refines the findings via a dedicated workspace. This guarantees high model accuracy, maintains Responsible AI governance, and ensures actionable items directly feed into CX operational enhancements.

Snowflake Cortex AI provides strong **out-of-the-box (OOTB)** functionality for many of these requirements, but certain enterprise-grade or multi-agent orchestrations require **custom builds** or hybrid configurations.

Here is a breakdown comparing what Snowflake Cortex delivers natively versus where custom tools are necessary:

1. Evaluation Framework (Evals)

- **Out-of-the-Box:**

-

- **Cortex Agent Evaluations / AI Observability:** Provides built-in **LLM-as-a-Judge** scoring for metrics such as *Answer Correctness*, *Logical Consistency*, *Groundedness*, and *Tool Selection Accuracy*.
-
- **NL Search Quality:** Functions like **Cortex Analyst** offer built-in semantic evaluation against structured data schemas.
-

- **Requires Custom Build:**

-

- **Custom Domain Rubrics:** If you have specific CX compliance rules or custom NPS prediction accuracy scoring, you need to build custom prompt rubrics using Snowflake's custom evaluation extension or external eval tools (e.g., Ragas, TruLens).
-

2. Observability & Operational Tracing

- **Out-of-the-Box:**

-

- **OpenTelemetry Tracing:** Snowflake Cortex automatically captures thread execution, agent steps, tool calls, and duration in **SNOWFLAKE.LOCAL.AI_OBSERVABILITY_EVENTS**.
-
- **Cost & Token Tracking:** Cost and usage metrics for Cortex LLM functions can be queried directly via Snowflake's **ACCOUNT_USAGE** and metering views.
-

- **Requires Custom Build:**

- **External Orchestration Tracing:** If multi-agent execution happens *outside* Snowflake (e.g., in LangGraph, CrewAI, or Python services running on AWS/Azure), Snowflake cannot capture those internal execution spans automatically. You will need a custom OpenTelemetry collector or platform (e.g., LangSmith, Phoenix) to trace external agents.

3. Guardrails & Governance

- **Out-of-the-Box:**

-

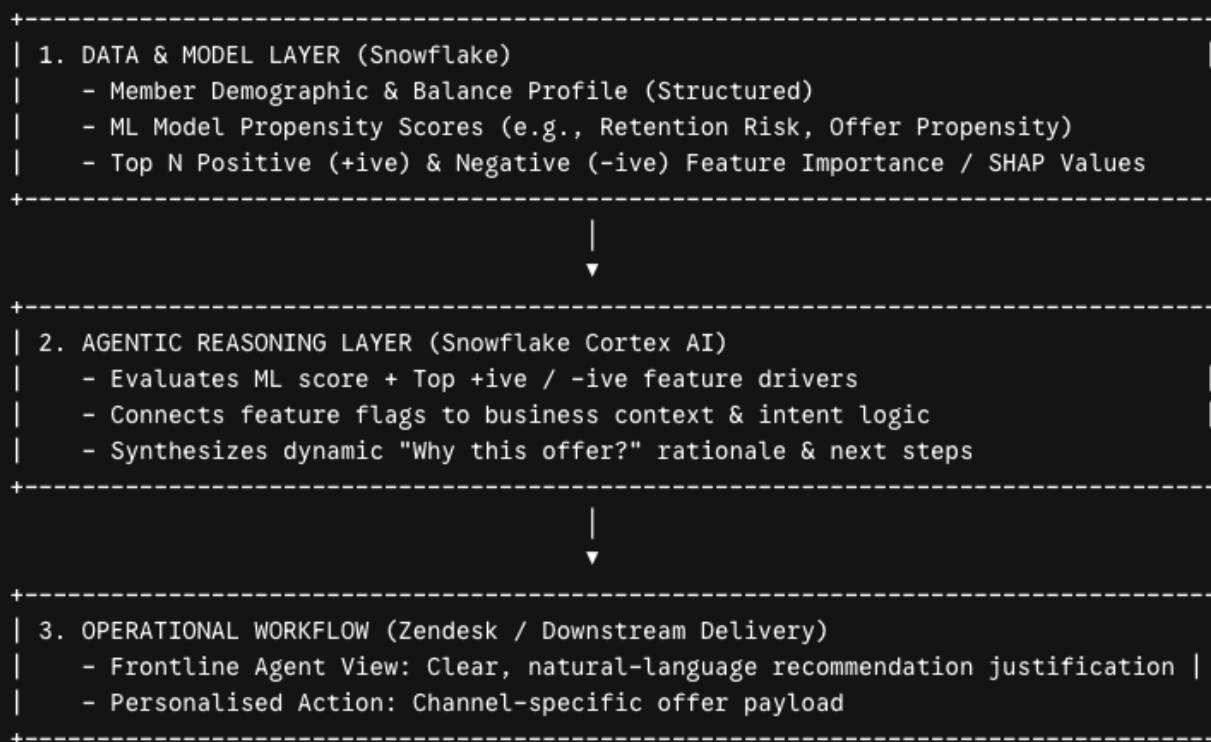
- **Data Security & PII Masking:** Snowflake natively handles data privacy via **Dynamic Data Masking**, **Row-Level Access Policies**, and RBAC (Role-Based Access Control).

-
- **Cortex Guard:** Includes built-in safety filters to detect and block unsafe content or responses.
-
- **Requires Custom Build:**
-
- **Pre-LLM Dynamic Redaction:** If you need real-time, context-aware PII redaction *before* unstructured verbatims are passed into LLM prompts (rather than static database column masking), you may need custom Python UDFs or customized pre-processing logic in your pipeline.

Summary Matrix

Capability	Out-of-the-Box (Snowflake Cortex)	Requires Custom Build / Hybrid
Evaluations	Standard metrics (Groundedness, Correctness)	Custom business rules & domain-specific rubrics
Observability	In-Snowflake agent execution & OpenTelemetry spans	Tracing across multi-agent tools hosted outside Snowflake
Governance	Column masking, RBAC, Cortex Guard	Dynamic PII scrubbing for raw unstructured verbatims

- **AI Decisioning & Personalisation:** Led the design and delivery of an AI/ML-driven decisioning capability using member behavioural signals and multi-agent orchestration to generate channel-specific recommendations. An initial Zendesk PoC delivered a **10x uplift in recommendation conversion**, demonstrating the value of embedding AI-driven decisioning into operational workflows.



Agent Reasoning & Synthesis (Cortex AI Prompt):

The agent receives these raw structured inputs and synthesizes a 2-sentence operational summary for a Zendesk representative during a rare call, or formats an outbound direct payload:

Zendesk Frontline Guidance:

*"Member is near preservation age with a high balance, making them a strong candidate for pre-retirement planning (+ive drivers). However, they have zero portal activity in 12 months (-ive driver). **Recommended Action:** Offer a low-friction, guided call with a retirement specialist rather than directing them to online self-serve tools."*

🎯 Why This Architecture Wins

1. **Bridges the "Explainability Gap":** Frontline staff don't care about a float value like `0.82`. They care about *why* the system recommends an action. The agent connects the feature dots to explain the *why*.

2. **Handles "Passive" Member Profiles Intelligently:** Even if the member has zero unstructured conversation history, the agent uses the **-ive feature flags** (e.g., low digital usage) to adjust the delivery strategy (e.g., direct mail or assisted call vs. push notification).
3. **No LLM Hallucinations on Math:** The underlying ML model does the heavy lifting on predictions and feature math in Snowflake, while the LLM agent purely handles context translation and narrative generation.



Resume & Interview Bullet Point Refinement

Here is how you can update this achievement on your resume or talk about it in interviews:

*"Architected a hybrid AI decisioning framework in Snowflake, combining ML propensity scoring with Cortex AI Agents to drive Next Best Action. Leveraged model feature importance (top positive/negative SHAP values) within an agentic reasoning prompt to automatically synthesize natural-language offer justifications for frontline teams, delivering a **10x uplift in recommendation conversion** on Zendesk."*

Choice B

To implement this cleanly inside Snowflake, **you do not need separate .md files or external skill frameworks** (like semantic kernel plugins or agent skill directories).

Since you are operating directly within Snowflake using Snowpark / Python, you can achieve this with a single, highly structured call to `snowflake.cortex.Complete()` (or by wrapping it in a Snowflake User-Defined Function / Stored Procedure).

PDF

How to Implement: System Context + Feature Drivers Pattern

You pass the **System Context**, the **Model Output (+ive / -ive features)**, and the **Member Profile** directly into the `Complete()` prompt as a structured context block.

PDF

Python Implementation inside Snowflake Notebook

```
import json
from snowflake.cortex import Complete
```

```
def generate_nba_justification(
    member_id: str,
    recommended_offer: str,
    propensity_score: float,
    pos_features: list,
    neg_features: list,
    channel: str = "Zendesk"
```

```
) -> str:
    """
```

```
    Translates ML propensity scores and feature attributions into natural-language
    narrative reasoning for frontline staff.
    """
```

```
    # 1. System Prompt defining the Persona and Operational Rules
```

```
    system_prompt = """You are an AI Decisioning Assistant for a superannuation fund.
```

```
Your task is to review an ML model's recommendation along with its top positive (+ive) and
negative (-ive) feature drivers.
```

```
Synthesize these inputs into a 2-sentence 'Why this offer' justification and 'Suggested Action' for
a frontline service representative.
```

```
Rules:
```

- Be concise, professional, and actionable.
- Translate negative feature drivers (e.g. low digital engagement) into smart handling instructions (e.g. recommend human-assisted support over self-serve portal).
- Never mention raw metric names (like 'SHAP' or 'online_portal_logins_12m'). Use plain business language.

```
"""
```

```
    # 2. Context Payload constructed dynamically from your Snowflake Semantic View
```

```
    user_context = f"""
```

```
    [Member Context]
```

```
    Member ID: {member_id}
```

```
    Recommended Offer: {recommended_offer}
```

```
    Propensity Score: {propensity_score}
```

```
    Channel: {channel}
```

```
    [ML Feature Drivers]
```

```
    Top Positive Drivers (+ive): {' '.join(pos_features)}
```

```
    Top Negative Drivers (-ive): {' '.join(neg_features)}
```

"""

3. Combined Prompt for Cortex Complete

prompt = f"{system_prompt}\n\n{user_context}"

4. Single Cortex Complete Call

Using llama3.1-70b or mistral-large for strong reasoning capabilities

narrative = Complete("llama3.1-70b", prompt)

return narrative

--- EXAMPLE RUN ---

Pulling row data from your Snowflake Semantic View:

result = generate_nba_justification(

member_id="MEM-98742",

recommended_offer="Pre-Retirement Transition Strategy",

propensity_score=0.86,

pos_features=["Member age is 59", "Preservation age reached", "Super balance exceeds \$300k"],

neg_features=["Zero digital portal logins in 12 months", "No active advisor linked"],

channel="Zendesk"

)

print(result)

Expected Output Displayed on Zendesk / Operational Screen

Why this offer: *Member is approaching retirement age with a high balance, making them a prime candidate for a pre-retirement strategy (+ive drivers). However, they rarely use online portal tools (-ive driver).*

Suggested Action: *Discuss pre-retirement options verbally during the call and offer to arrange a callback with a retirement specialist rather than directing them to online forms.*

When would you actually need `.md` files or "Skills"?

If using Cortex Agents (The New Framework): If you transition from standard `Complete()` calls to the multi-tool **Snowflake Cortex Agents API**, Snowflake allows you to attach a **Semantic Model file** (`.yaml` or `.md`) to define table relations, metrics, and business logic so the agent can autonomously query the database.

If using LangChain / LangGraph: If building multi-turn agentic workflows external to native Cortex, developers often store system personas and tool definitions in `.md` or `.prompt` files for version control.

For your exact pipeline: Relying on `snowflake.cortex.Complete()` with a structured system context is the cleanest, lowest-latency, and most cost-effective architecture inside Snowflake.

- **Enterprise Knowledge & Agentic AI:** Designed and delivered an Agentic RAG solution using a vector-database-backed enterprise knowledge base to retrieve and synthesise complex product information within customer service workflows. Enabled grounded, actionable responses for staff and reduced average call handling time by **40%**.

